

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение
высшего образования

«КРЫМСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ им. В. И. ВЕРНАДСКОГО»
ФИЗИКО-ТЕХНИЧЕСКИЙ ИНСТИТУТ

Кафедра компьютерной инженерии и моделирования

ПЕРСОНАЛЬНЫЙ ФИНАНСОВЫЙ МЕНЕДЖЕР

Экзаменационный проект

по дисциплине «Современные технологии программирования»

студента 1 курса группы ПИ-б-о-241

Фамилия Имя Отчество

направления подготовки 09.03.04 «Программная инженерия»

Экзамен принимал

старший преподаватель кафедры

компьютерной инженерии и моделирования

(оценка)

(подпись, дата)

Чабанов В.В.

Симферополь, 2025

РЕФЕРАТ

Разработка мобильного приложения "Персональный финансовый менеджер". – Симферополь: ФТИ КФУ им. В. И. Вернадского, 2025. – 33 с., 9 ил., 3 ист.

Объект разработки – мобильное веб-приложение для учета личных финансов, формирования отчетов и анализа расходов.

Цель работы – создание удобного и безопасного инструмента для управления персональным бюджетом, ориентированного на пользователей, не имеющих специальных знаний в области финансов.

Рассмотрены аналогичные решения, их преимущества и недостатки, спроектирована архитектура приложения, выбран стек технологий для реализации. Разработан базовый функционал: модуль учета операций, система категорий, формирование отчетов в виде диаграмм, а также модули авторизации и работы с данными. Проведено тестирование ключевых функций, обеспечена безопасность хранения пользовательской информации. Описаны перспективы дальнейшего развития проекта, включая интеграцию с банковскими системами и расширенную аналитику.

ЛИЧНЫЕ ФИНАНСЫ, МОБИЛЬНОЕ ПРИЛОЖЕНИЕ, УЧЕТ РАСХОДОВ, ФИНАНСОВАЯ ГРАМОТНОСТЬ, ОТЧЕТЫ, БЕЗОПАСНОСТЬ ДАННЫХ, ВЕБ-ТЕХНОЛОГИИ

ОГЛАВЛЕНИЕ

РЕФЕРАТ.....	2
ОГЛАВЛЕНИЕ.....	3
ВВЕДЕНИЕ.....	4
ГЛАВА 1 ПОСТАНОВКА ЗАДАЧИ.....	5
1.1 Цель проекта.....	5
1.2 Существующие аналоги.....	5
1.3 Основные отличия от аналогов.....	8
1.4 Структура проекта.....	9
ГЛАВА 2 ПРОГРАММНАЯ РЕАЛИЗАЦИЯ.....	10
2.1 Стек технологий.....	10
2.2. Объектная модель предметной области.....	10
2.3 Структура модулей.....	12
2.4 Серверная часть (Back-end).....	13
2.5. Интеграция и API.....	15
2.6. Клиентская часть (Front-end).....	16
2.7. Пользовательский интерфейс.....	16
2.8. Основные структуры данных.....	20
ГЛАВА 3 СТРУКТУРА КОНВЕЙЕРА CI/CD.....	23
3.1 Тестирование исходного кода.....	23
3.2 Структура конвейера непрерывной интеграции.....	23
3.3 Структура конвейера непрерывной доставки и развёртывания.....	24
ГЛАВА 4 ПЕРСПЕКТИВЫ ДАЛЬНЕЙШЕГО РАЗВИТИЯ ПРОЕКТА.....	25
4.1 Перспективы технического развития.....	25
4.2 Перспективы монетизации.....	26
ЗАКЛЮЧЕНИЕ.....	28
ЛИТЕРАТУРА.....	30
ПРИЛОЖЕНИЕ 1 ТЕХНИЧЕСКОЕ ЗАДАНИЕ НА ВЫПОЛНЕНИЕ РАБОТ ПО ПРОЕКТУ.....	31

ВВЕДЕНИЕ

В современном мире мобильные приложения для управления финансами становятся все более востребованными. С развитием цифровых технологий и увеличением числа безналичных платежей у пользователей возникает потребность в удобных инструментах для учета личных расходов и доходов. Особенно это актуально для студентов и молодых специалистов, которые только начинают осваивать основы финансовой грамотности.

Предметом данной работы является разработка мобильного приложения "Персональный финансовый менеджер", предназначенного для простого и удобного учета личных финансов. Основная цель приложения - предоставить пользователям интуитивно понятный инструмент для контроля своих расходов без необходимости изучения сложных финансовых терминов и методик.

Актуальность работы обусловлена тем, что существующие аналоги либо слишком сложны для начинающих пользователей, либо требуют предоставления доступа к банковским счетам, что вызывает опасения у многих людей.

Для достижения поставленной цели необходимо решить следующие задачи:

- Проанализировать существующие аналогичные приложения и выявить их сильные и слабые стороны
- Определить оптимальный набор функций для базовой версии приложения
- Разработать простой и удобный пользовательский интерфейс
- Реализовать основные модули приложения: учет операций, категории расходов, формирование отчетов
- Обеспечить безопасное хранение пользовательских данных
- Провести тестирование работоспособности приложения

ГЛАВА 1

ПОСТАНОВКА ЗАДАЧИ

1.1 Цель проекта

Целью проекта является разработка веб-приложения «Персональный финансовый менеджер», предназначенного для анализа финансовых транзакций, управления личными финансами и оценки экологического воздействия расходов пользователя.

Проект направлен на повышение финансовой грамотности пользователей и снижение негативного воздействия на окружающую среду через осознанное управление расходами.

1.2 Существующие аналоги

В процессе разработки были проанализированы следующие аналоги: Кошелёк (<https://koshelek.ru>) представляет собой приложение для управлением личными финансами. Его основные преимущества включают бесплатное использование, напоминания о платежах и возможность планирования бюджета. Однако у сервиса есть и недостатки: ограниченные возможности аналитики и удобный, простой интерфейс со всеми функциями есть только в мобильной версии, что сужает аудиторию пользователей.

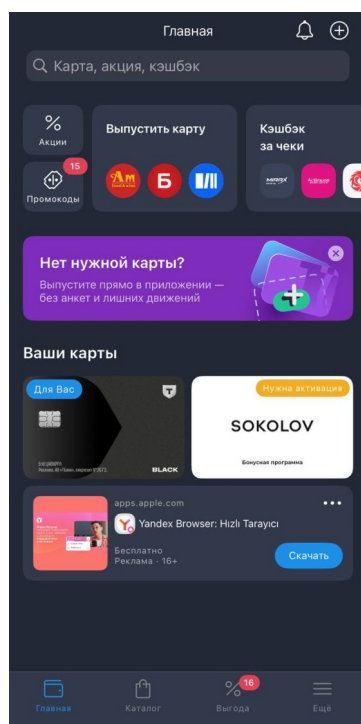


Рисунок 1.2.1 – Главная страница

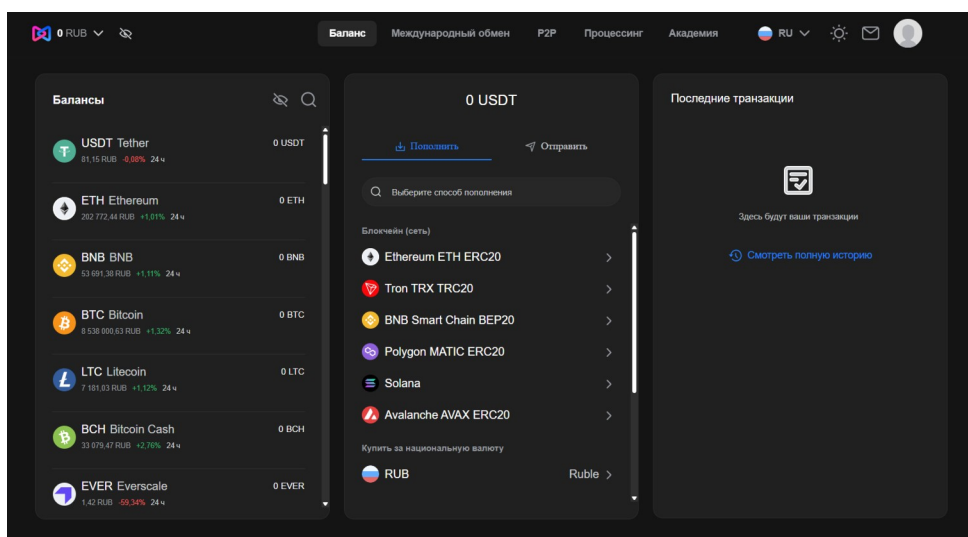


Рисунок 1.2.2 – Страница баланса в веб-версии

Дребеденьги (<https://drebedengi.ru>) предлагает более продвинутые функции, такие как поддержка совместного использования для семьи, гибкие отчёты и интеграция с банками. Тем не менее, сервис требует платной подписки для доступа к основным возможностям, имеет устаревший дизайн и сложен в настройке для новичков, что может отпугнуть часть потенциальных пользователей.

Рисунок 1.2.3 – Страница расходов

Меню

Ввод операций

Планирование бюджета

Накопления, список покупок

Графики, отчёты, история

Справочники

Другое

Оплата сервиса

Ввод операций

Присылайте файлы с расходами, обрабатываем.

План бюджета Семейный, ежемесячный, в руб.

Скрыть панель

Июнь 2025

План	Факт
-57 100 руб.	-9 573 руб.
Осталось: 47 526 руб.	

Июль 2025

План	Факт
-57 100 руб.	0 руб.
Осталось: 0 руб.	

Август 2025

План	Факт
-58 300 руб.	0 руб.
Осталось: 0 руб.	

Сентябрь 2025

План	Факт
-57 100 руб.	0 руб.
Осталось: 0 руб.	

Октябрь 2025

План	Факт
-57 100 руб.	0 руб.
Осталось: 0 руб.	

Ноябрь 2025

План	Факт
-58 300 руб.	0 руб.
Осталось: 0 руб.	

Расходы

Еда

Дом, семья

Квартплата

Автомобиль

Квартира

Доходы

Фазерлинг

Банковские проценты

Остатки

Рисунок 1.2.4 – Планирование бюджета

Mint (<https://mint.com>) – один из лидеров на международном рынке финансовых приложений. Его сильные стороны – автоматическая синхронизация с банком, удобная аналитика с детализированными отчетами и бесплатный доступ к базовым функциям. Однако сервис ориентирован преимущественно на пользователей из США, так как поддержка банков других стран ограничена. Кроме того, бесплатная версия содержит рекламу, а интерфейс может показаться перегруженным для тех, кто только начинает разбираться в управлении финансами.

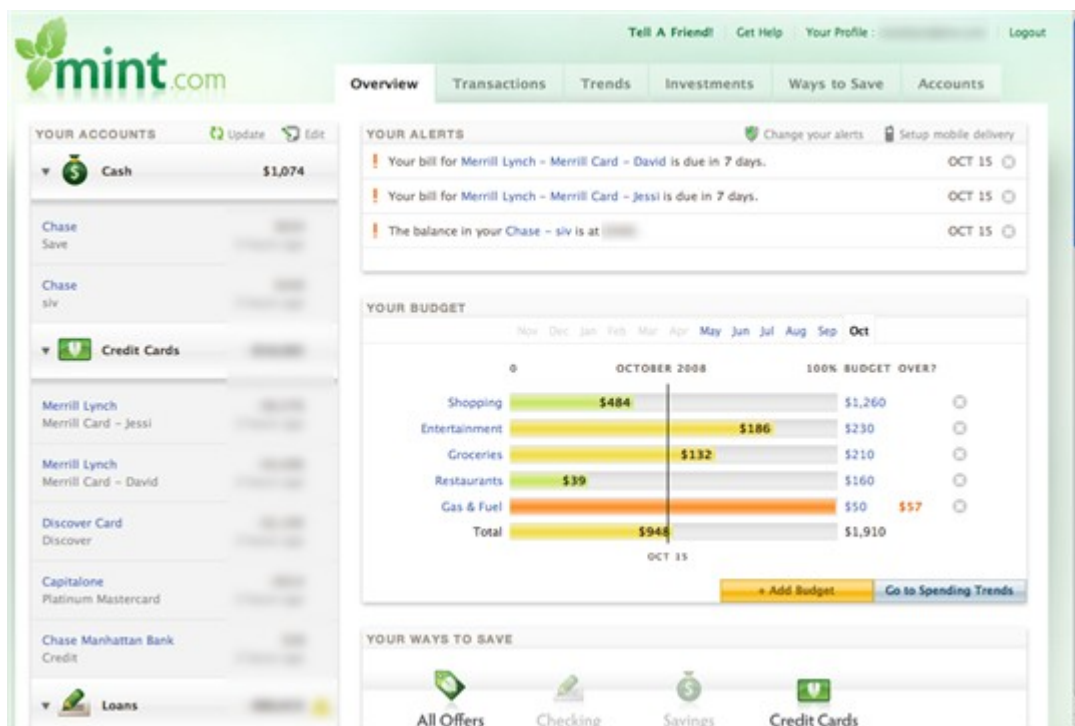


Рисунок 1.2.5 – Обзорная страница

1.3 Основные отличия от аналогов

Наш финансовый менеджер предлагает ряд преимуществ по сравнению с аналогами. Главное достоинство – исключительная простота и удобство использования. Мы создали интуитивно понятный интерфейс, который позволяет мгновенно начать работу без длительного изучения функций. В отличие от перегруженных сложными настройками аналогов, наш сервис фокусируется на действительно важных для пользователя возможностях. Мы сделали ставку на ручной ввод операций – это не только просто в реализации, но и безопаснее для пользователей. Все необходимые функции – удобный учет расходов и доходов, наглядные диаграммы по категориям – реализованы максимально просто и понятно.

Хотя в текущей версии нет некоторых продвинутых возможностей, имеющийся функционал покрывает основные потребности пользователей, желающих контролировать свои финансы без лишних сложностей. Простота использования и понятность интерфейса делают наше решение отличным

выбором для тех, кто ценит свое время и хочет вести учет расходов максимально комфортно.

1.4 Структура проекта

Проект состоит из нескольких модулей:

1. Модуль аутентификации и авторизации: отвечает за регистрацию новых пользователей, вход в систему и защиту личных данных. Контролирует, кто может просматривать и изменять информацию.

2. Модуль управления пользователями: хранит и обновляет данные пользователей: личную информацию, настройки и финансовые цели. Связывает эти данные с другими разделами приложения.

3. Модуль обработки транзакций и финансовых данных: учитывает все доходы и расходы, сортирует их по категориям и подготавливает для создания отчетов.

4. Модуль аналитики и отчетности: анализирует финансовые данные, создает наглядные графики и дает рекомендации по управлению бюджетом.

5. Модуль управления финансовыми целями: позволяет создавать и отслеживать личные финансовые цели, взаимодействует с аналитикой для оценки прогресса.

6. Frontend-модуль: простое и удобное приложение для управления деньгами, просмотра отчетов и работы с целями.

7. Backend API: обрабатывает бизнес-логику, запросы от frontend и интеграции. Обеспечивает безопасность данных и стабильность работы системы.

ГЛАВА 2

ПРОГРАММНАЯ РЕАЛИЗАЦИЯ

2.1 Стек технологий

React — построение компонентного и реактивного интерфейса

TypeScript — строгая типизация, упрощение поддержки и масштабирования

Tailwind CSS — быстрая стилизация без кастомного CSS

Radix UI — готовые доступные компоненты интерфейса

shadcn/ui — ускорение разработки с помощью готовых UI-блоков

Node.js — асинхронная среда выполнения серверного кода

Express — минималистичный фреймворк для создания REST API

Vite — быстрая сборка и горячая перезагрузка

esbuild — высокая скорость сборки серверной части

tsx — удобный запуск TypeScript без компиляции

drizzle-orm — ORM с полной поддержкой типов и миграций

pg — подключение и работа с PostgreSQL

@neondatabase/serverless — серверлесс-доступ к PostgreSQL

passport — аутентификация пользователей

passport-local — стратегия входа по логину и паролю

express-session — хранение сессий на сервере

bcrypt — безопасное хеширование паролей

jsonwebtoken — генерация и проверка JWT

react-hook-form — эффективное управление формами

zod — типобезопасная и декларативная валидация данных

2.2. Объектная модель предметной области

Система состоит из одной базы данных, которая включает в себя несколько взаимосвязанных таблиц:

— users — таблица, хранящая информацию о пользователях системы. Каждая запись содержит уникальный идентификатор пользователя (id), электронную

почту (email), пароль (password), имя (first_name), фамилию (last_name), а также дату и время создания (created_at) и последнего обновления (updated_at) записи.

– transactions – таблица, содержащая данные о финансовых операциях пользователей. Каждая транзакция связана с пользователем через внешний ключ user_id. Транзакция характеризуется типом (type), который может быть «income» (доход) или «expense» (расход), суммой (amount), категорией (category), описанием (description), датой операции (date), а также дополнительным параметром экологического воздействия (eco_impact). Также в таблице фиксируется время создания записи (created_at).

– categories – справочник категорий транзакций, принадлежащих конкретному пользователю (через внешний ключ user_id). Категории разделяются по типу — доход или расход (type), имеют название (name), цветовое обозначение (color) и иконку (icon) для удобства отображения в интерфейсе. Кроме того, категория может иметь лимит бюджета (budget_limit). Время создания категории хранится в поле created_at.

Как видно, таблицы transactions и categories связаны с таблицей users через внешний ключ user_id, что обеспечивает учет и разделение данных по пользователям.

Использование отдельной таблицы категорий позволяет централизованно управлять наименованиями и параметрами категорий, что способствует:

– Защите от некорректного ввода: пользователю предлагается выбирать категории из уже существующих, что снижает риск ошибок и дублирования.

– Упрощению обновления данных: изменение параметров категории (например, названия или цвета) автоматически применяется ко всем связанным транзакциям. На рисунке 2.2.1 представлена структура базы данных системы, демонстрирующая взаимосвязи между таблицами пользователей, транзакций и категорий.

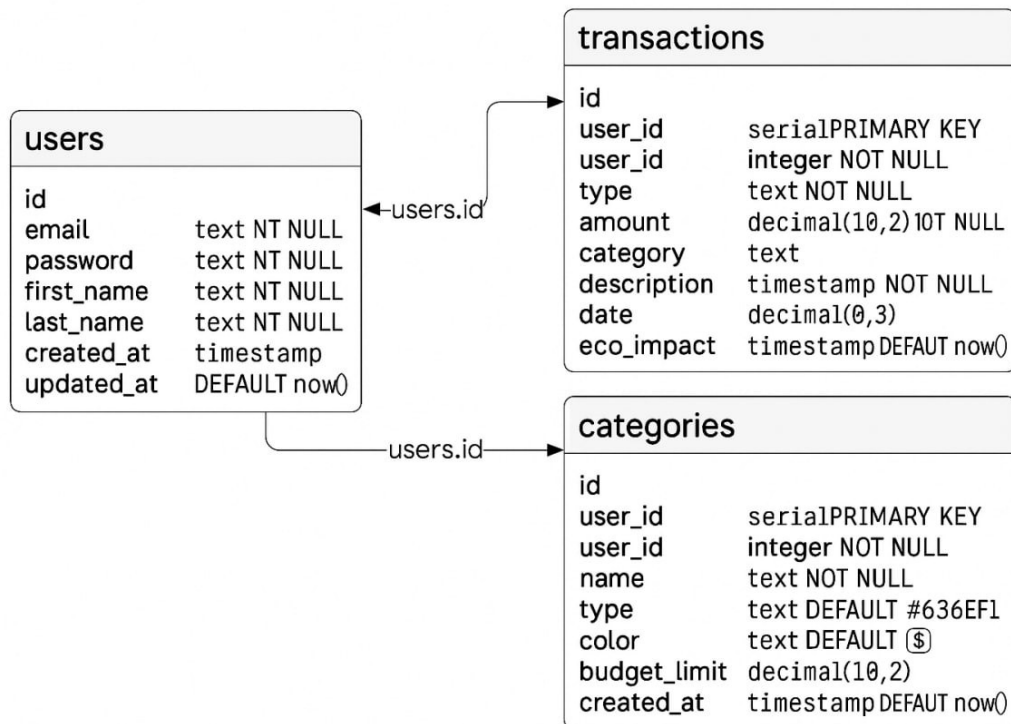


Рисунок 2.2.1 Структура базы данных

2.3 Структура модулей

1. Модуль аутентификации и авторизации: обеспечивает регистрацию, вход пользователей и защиту доступа с помощью JWT. Взаимодействует с backend API и frontend для контроля доступа к личным данным.

2. Модуль управления пользователями: отвечает за хранение и обновление профилей пользователей, их настроек и финансовых целей. Получает данные от модуля аутентификации и передает их в модули аналитики и отчетности.

3. Модуль обработки транзакций и финансовых данных: классифицирует и хранит финансовые операции, подготавливая данные для аналитики. Взаимодействует с базами данных и модулем аналитики.

4. Модуль аналитики и отчетности: формирует графики, отчеты и рекомендации на основе транзакций и целей пользователя. Обеспечивает визуализацию данных во frontend и интеграцию с модулем целей.

5. Модуль управления финансовыми целями: позволяет создавать и отслеживать личные финансовые цели, взаимодействует с аналитикой для оценки прогресса.

6. Frontend-модуль (React): предоставляет пользователю удобный интерфейс для управления финансами, просмотра отчетов и настройки целей. Обменивается данными с backend через API, защищенные JWT.

7. Backend API (Node.js): управляет бизнес-логикой, принимает запросы от frontend, взаимодействует с базами данных, обеспечивает безопасность и стабильность работы всего приложения.

2.4 Серверная часть (Back-end)

1. Архитектура и назначение

Серверная часть реализована как RESTful API на Node.js с использованием Express.js. Отвечает за:

- Аутентификацию и авторизацию пользователей (JWT).
- Обработку и хранение финансовых данных.
- Интеграцию с банковскими API для импорта транзакций.
- Логику формирования аналитики и отчетов.
- Управление пользовательскими настройками и целями.
- Кэширование и оптимизацию работы через Redis.
- Обеспечение безопасности и производительности.

2. Требования к оборудованию

- **Процессор:** минимум 4 ядра (Intel Xeon E3 или аналог).
- **Оперативная память:** не менее 8 ГБ (оптимально 16 ГБ).
- **Хранилище:** SSD с минимум 100 ГБ свободного места (для хранения логов, базы данных и кэшей).
- **Сеть:** стабильное подключение с пропускной способностью не менее 100 Мбит/с.
- **Резервирование:** наличие RAID-массива или облачные решения с автоматическим бэкапом.

3. Операционная система

- Рекомендуется использовать Linux-сервер (Ubuntu Server 22.04 LTS или аналог — Debian, CentOS).
- Обеспечивает стабильность, безопасность и совместимость с используемыми технологиями.

4. Программные зависимости и стек

- **Node.js** (версия 18.x или выше) — среда выполнения backend.
- **Express.js** — фреймворк для API.
- **PostgreSQL** — основная реляционная база данных для хранения структурированных данных.
- **MongoDB** — возможно, для хранения неструктурированных данных (например, логов или кэшированных документов).
- **Redis** — кэширование, хранение сессий и ускорение ответов API.
- **Nginx** — обратный прокси и балансировщик нагрузки, обеспечивает SSL-терминацию и статическую выдачу.
- **JWT (JSON Web Tokens)** — для управления сессиями и безопасности.
- **Git** и **GitHub** — для управления версиями и развертывания.
- **Jira** — система управления задачами и трекинга багов.

5. Способы администрирования

- **Прямой доступ:** физический доступ к серверу для обслуживания и аварийного восстановления.
- **Удалённый доступ:** SSH (протокол безопасного терминального доступа), с двухфакторной аутентификацией и ограничениями по IP.
- **Веб-интерфейс:**
 - Панель мониторинга (например, Grafana/Prometheus) для наблюдения за состоянием сервера и нагрузкой.
 - Веб-интерфейс управления (например, Portainer для Docker, если используется контейнеризация).
- **Автоматизированное администрирование:** CI/CD через GitHub Actions или Jenkins для автоматического развертывания и обновления.

- **Резервное копирование:** автоматические бэкапы баз данных и конфигураций с возможностью восстановления через веб-интерфейс или скрипты.

2.5. Интеграция и API

1. Наличие внешнего API

Серверная часть предоставляет внешний RESTful API для взаимодействия с клиентским приложением и сторонними сервисами.

2. Основные возможности API:

- **Аутентификация и авторизация:**
 - Регистрация пользователей.
 - Вход с использованием JWT-токенов.
 - Восстановление пароля.
- **Работа с финансовыми данными:**
 - Получение списка транзакций пользователя.
 - Добавление, редактирование и удаление транзакций.
 - Импорт транзакций из банков через интеграцию с банковскими API.
- **Управление бюджетом и целями:**
 - Создание и обновление финансовых целей.
 - Получение текущего состояния бюджета.
 - Получение рекомендаций по бюджету и достижению целей.
- **Отчеты и аналитика:**
 - Запрос еженедельных и месячных отчетов.
 - Получение статистики расходов по категориям.
 - Доступ к графикам и диаграммам через API.
- **Административные функции:**
 - Управление профилем пользователя.
 - Настройки уведомлений и интеграций.

3. Технические особенности API:

- Формат передачи данных — JSON.
- Версионирование API (например, /api/v1/).
- Защита данных через HTTPS и аутентификацию JWT.
- Ограничение количества запросов (rate limiting) для предотвращения злоупотреблений.

2.6. Клиентская часть (Front-end)

Требования к клиентскому устройству:

- ОС: Windows 10+, macOS 11+, Linux (современные дистрибутивы)
- Браузер: Chrome, Firefox, Safari или Edge последних версий
- Оборудование: от 4 ГБ ОЗУ, желательно 8 ГБ; процессор с 2+ ядрами
- Разрешение экрана: от 1280×720

Программные зависимости на клиенте:

- React 18
- React DOM
- Radix UI компоненты (меню, модалки, формы и т. д.)
- React Hook Form + @hookform/resolvers + zod для валидации
- Framer Motion, clsx, lucide-react, chart.js, recharts, react-icons
- Tailwind CSS, tailwindcss-animate, tw-animate-css, tailwind-merge
- Wouter — маршрутизация без react-router

2.7. Пользовательский интерфейс

Общие принципы дизайна

Пользовательский интерфейс EcoFinance реализован в минималистичном современном стиле с фокусом на:

- **Интуитивности:** логичная группировка элементов, предсказуемая навигация

- Доступности: контрастные цвета, адаптивная верстка (поддержка мобильных устройств)
 - Визуальной ясности: использование иконок, прогресс-баров и цветовой дифференциации
 - Минимализме: отсутствие лишних элементов, фокус на функциональности
-

Ключевые элементы интерфейса

Навигационная панель (всегда доступна в шапке):

- Логотип EcoFinance (возврат на главную)
- Иконки основных разделов: Главная | Транзакции | Отчеты | Профиль
- Индикатор текущего баланса (справа)

Карточки данных (используются на главной):

- Финансовые метрики (баланс, доходы/расходы)
- Эко-рейтинг с цветовой индикацией
- Прогресс-бары для целей

Интерактивные элементы управления:

- Кнопка + Добавить (транзакцию/цель)
- Фильтры по периодам и категориям
- Выпадающие меню действий (:)
- Переключатели типа операций (доход/расход)

Сценарии взаимодействия

1. Процесс регистрации нового пользователя

- 1) На публичной странице пользователь нажимает кнопку "Регистрация".
- 2) Открывается форма с полями: email, пароль, имя и фамилия.
- 3) После ввода данных система проверяет валидность email и сложность пароля.
- 4) После подтверждения происходит автоматический вход в систему.
- 5) Пользователь заносится в базу данных.

6) Нового пользователя перенаправляет на главный дашборд.

Элементы формы:

- Валидация email в реальном времени.
- Индикатор сложности пароля.
- Чекбокс согласия с условиями использования.
- Визуальная обратная связь при ошибках.

2. Процесс добавления транзакции

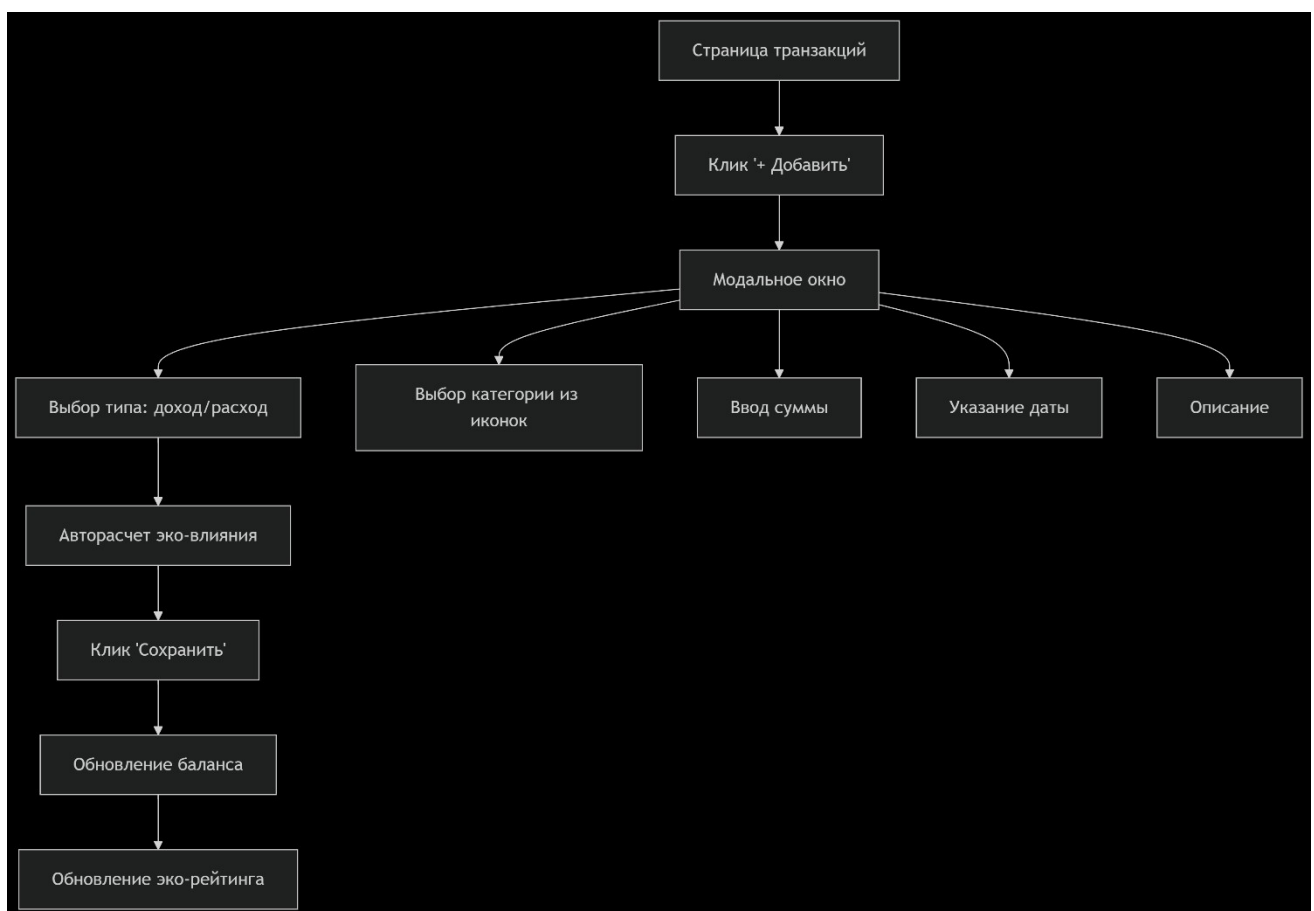


Рисунок 2.7.1 – Схема добавления транзакциями

Элементы интерфейса:

- Визуальный пикер категорий с иконками.
- Автоподстановка частых операций.
- Preview эко-воздействия в кг CO₂.
- Интерактивный календарь.

3. Анализ финансовых отчетов

- 1) В разделе "Отчеты" пользователь выбирает временной период.
- 2) Система предлагает настроить параметры отчета (тип данных, категории).
- 3) Автоматически генерируются визуализации: круговые диаграммы, графики.
- 4) Пользователь анализирует распределение расходов и экологическое воздействие.

Доступны опции:

- Экспорт данных в Excel
- Сравнение с предыдущими периодами
- Детализация по категориям

Особенности интерфейса:

- Интерактивные диаграммы с подсказками при наведении
- Двойная визуализация (финансы + экология)
- Слайдер для сравнения периодов
- Кнопка мгновенного экспорта данных

4. Процесс управления финансовыми целями



Рисунок 2.7.2 – Схема управления финансовыми целями

Элементы управления:

- Прогресс-бар с % выполнения.
- Индикатор оставшегося времени.

- Кнопка "Внести средства".
- Тултипы с рекомендациями.

2.8. Основные структуры данных

Поскольку в проекте используется ORM-фреймворк drizzle-orm для работы с базой данных PostgreSQL, одной из основных групп структур, используемых в программной реализации, являются DTO (Data Transfer Object). Data Transfer Object — это структуры, которые повторяют схему таблиц в базе данных и используются либо для получения данных, которые преобразуются в объекты, либо для сохранения новых записей путём отправки соответствующих данных в БД.

DTO представлены следующими структурами:

- users – таблица, описывающая пользователей. Содержит следующие поля: id, email, password, firstName, lastName, createdAt, updatedAt. Все поля обязательные, кроме updatedAt, который заполняется автоматически;
- transactions – таблица, отражающая финансовые транзакции пользователя. Содержит поля: id, userId (внешний ключ), type (значение 'income' или 'expense'), amount, category, description (необязательное), date, ecoImpact (необязательное), createdAt;
- categories – таблица пользовательских категорий. Включает поля: id, userId (внешний ключ), name, type (также 'income' или 'expense'), color (по умолчанию — #6366F1), icon (по умолчанию — 💰), budgetLimit (опционально), createdAt.

Связи между таблицами определяются с помощью relations:

- usersRelations – один пользователь может иметь множество транзакций и категорий;
- transactionsRelations – каждая транзакция относится к одному пользователю;
- categoriesRelations – каждая категория также принадлежит одному пользователю.

Для приёма и валидации данных, получаемых от клиента, а также формирования ответов от сервера, используются бизнес-структуры и схемы на основе zod:

- `insertUserSchema` – схема, применяемая при регистрации нового пользователя. Включает поля `email`, `password`, `firstName`, `lastName`;
- `loginSchema` – используется для авторизации пользователя. Валидация включает формат e-mail и минимальную длину пароля (6 символов);
- `insertTransactionSchema` – применяется при создании транзакции. Обязательные поля: `type`, `amount`, `category`, `date`. Дополнительно можно указать `description`;
- `insertCategorySchema` – используется при создании новой категории. Включает поля `name`, `type`, `color`, `icon`, `budgetLimit`.

В проекте также определены типы данных, получаемые на основе описанных схем:

- `User`, `Transaction`, `Category` – типы, соответствующие выборке данных из базы (`$inferSelect`);
- `InsertUser`, `LoginUser`, `InsertTransaction`, `InsertCategory` – типы, определяемые на основе схем валидации `zod`.

Все операции с базой данных инкапсулированы в одном модуле подключения, который использует `pg.Pool` и `drizzle()` с передачей схемы:

- Подключение настраивается через `.env`, параметры считываются с использованием `dotenv`;
- В качестве пула соединений используется `pg`, а ORM инициализируется функцией `drizzle(pool, { schema })`.

Для приёма HTTP-запросов используется фреймворк `Express.js`. Он обрабатывает входящие запросы, логирует действия и маршруты, а также выводит JSON-ответы. В проекте присутствует централизованный `middleware` для логирования запросов, а также глобальный обработчик ошибок.

- Запросы, начинающиеся с `/api`, логируются с фиксированием метода, пути, статуса и времени выполнения;
- В случае ошибки, сервер возвращает JSON-объект с полем `message` и соответствующим HTTP-статусом;

– В зависимости от режима (development или production) приложение подключает Vite или отдаёт статику.

Таким образом, структура данных проекта полностью отделена от логики, чётко типизирована и сопровождается надёжной системой валидации и логирования.

ГЛАВА 3

СТРУКТУРА КОНВЕЙЕРА CI/CD

3.1 Тестирование исходного кода

Инструменты тестирования:

Клиент (Frontend):

- Jest + React Testing Library (для React-приложений)
- npm test – запуск unit- и интеграционных тестов

Сервер (Backend):

- Jest/Mocha + Supertest (для API)
- npm test – проверка основных модулей

Что не покрыто тестами:

- Интеграции со сторонними API (мокируются)
- UI-анимации и сложные взаимодействия
- Крайние случаи в обработке платежей

3.2 Структура конвейера непрерывной интеграции

Инструменты:

Git (GitHub / GitLab / Bitbucket) – хранение исходного кода и триггеры CI.

GitHub Actions – оркестрация CI-пайплайна.

Docker – сборка и контейнеризация приложения.

Docker Hub / GitHub Container Registry – хранение образов Docker.

Этапы:

1. Запуск: При пуше в main/dev или создании PR.
2. Кэширование: Ускоряет установку зависимостей.
3. Тесты:
 - Клиент: Jest + проверка покрытия.
 - Сервер: Модульные тесты API.
4. Артефакты: Отчёты о покрытии сохраняются в GitHub Actions.

3.3 Структура конвейера непрерывной доставки и развёртывания взаимодействует.

Этапы CI:

Запуск пайплайна при push / PR в ветки main / develop.

Запуск тестов

Сборка Docker-образа (если тесты пройдены).

Публикация образа в Docker Hub (с тегом latest для main, dev для develop).

Этапы:

1. Сборка:
 - Создание Docker-образов для клиента и сервера.
 - Теги: latest (для основного образа) + git-SHA (для откатов).
2. Публикация: Отправляет образы в Docker Hub.
3. Деплой: Через SSH обновляет контейнеры на сервере с помощью docker-compose.

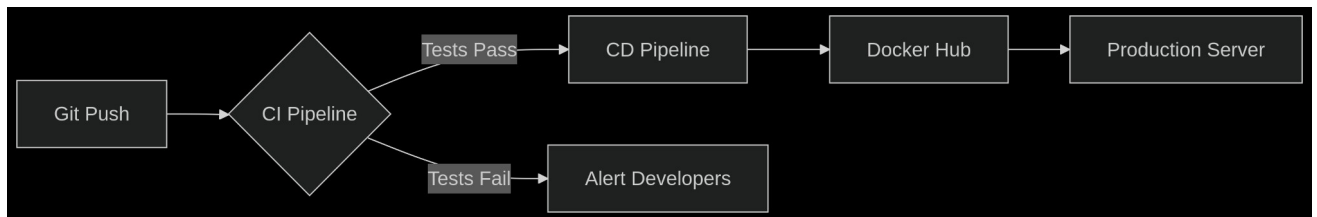


Рисунок 3.3.1 – Схема непрерывной доставки и развёртывания

ГЛАВА 4

ПЕРСПЕКТИВЫ ДАЛЬНЕЙШЕГО РАЗВИТИЯ ПРОЕКТА

4.1 Перспективы технического развития

Проект «Персональный финансовый менеджер» изначально разрабатывается с учетом возможности дальнейшего масштабирования и модернизации, что открывает широкие перспективы его технического развития.

Одной из ключевых задач дальнейшего развития является реализация мобильного приложения как для платформ Android, так и iOS. Это позволит пользователям получать доступ к своему финансовому состоянию в любое время и в любом месте, а также улучшит удобство использования сервиса по сравнению с аналогами, которые имеют ограниченную или отсутствующую веб-поддержку.

Для повышения функциональности планируется расширение поддержки банковских API. На начальном этапе интеграция будет осуществляться с крупными российскими банками, но в будущем возможно добавление поддержки международных финансовых организаций, что значительно увеличит целевую аудиторию продукта.

Также предусмотрено внедрение многоязычности интерфейса. На данном этапе приложение будет работать только на русском языке, однако уже сейчас закладываются основы для перевода на английский и другие популярные языки. Это сделает продукт привлекательным для пользователей за пределами России.

Еще одной важной технической доработкой станет поддержка нескольких валют. В текущей версии система работает только с рублем, но в дальнейшем планируется добавить возможность автоматического конвертирования валют по рыночным курсам, что особенно важно для пользователей, ведущих международные расчеты или путешествующих.

Для улучшения аналитических возможностей системы предполагается интеграция искусственного интеллекта (AI), который сможет давать персонализированные рекомендации по оптимизации расходов, формированию

бюджета и достижению финансовых целей. Такая функциональность выделит проект среди конкурентов и повысит лояльность пользователей.

Кроме того, будет совершенствоваться архитектура backend: переход на микросервисную модель позволит повысить отказоустойчивость, упростить масштабирование и обновление отдельных компонентов без остановки работы всей системы. Также планируется более активное использование Redis для кэширования данных, что ускорит работу приложения и снизит нагрузку на базу данных PostgreSQL.

Таким образом, перспективы технического развития проекта включают создание мобильной версии, расширение географии поддерживаемых банков, внедрение многоязычности и мультивалютности, интеграцию ИИ и модернизацию серверной архитектуры.

4.2 Перспективы монетизации

Несмотря на то, что первоначальная версия приложения будет предоставляться бесплатно, проект имеет четко проработанную стратегию монетизации, которая позволит обеспечить устойчивое развитие продукта и покрытие затрат на его обслуживание и развитие.

На начальном этапе планируется использовать Premium-модель, при которой базовые функции будут доступны всем пользователям, а расширенные возможности станут доступны подписчикам Premium-тарифа. К таким возможностям могут относиться:

- Расширенная аналитика и прогнозирование трат;
- Поддержка большего количества банковских счетов;
- Возможность создания неограниченного числа финансовых целей;
- Отсутствие рекламы;
- Экспорт отчетов в формате PDF/Excel;
- Доступ к ИИ-рекомендациям и прогнозам.

Кроме того, можно рассмотреть партнерские программы с банками и финансовыми организациями. Например, приложение может предлагать пользователям выгодные условия по кредитам, депозитам или инвестиционным

продуктам, получая комиссию за каждого привлеченного клиента. Такой подход позволяет монетизировать продукт без ущерба для пользовательского опыта.

Еще одним направлением монетизации является интеграция рекламы. В бесплатной версии приложения может быть реализована ненавязчивая баннерная реклама, связанная с финансовыми услугами, инвестициями или образованием. Однако, чтобы сохранить положительный пользовательский опыт, реклама должна быть контекстной и легко игнорируемой.

Потенциальным источником дохода может стать предоставление корпоративных лицензий. Например, компания может приобрести доступ к специальной версии приложения для своих сотрудников с возможностью настройки под бренд компании и интеграции в корпоративную систему управления персоналом.

Также возможна реализация модели SaaS (Software as a Service) — когда приложение предлагается как облачный сервис с ежемесячной или годовой подпиской, с возможностью централизованного управления, аналитики и отчетности.

В перспективе, при значительном росте пользовательской базы, можно рассматривать продажу агрегированных и обезличенных данных (например, статистики по категориям расходов) аналитическим агентствам или исследовательским организациям, соблюдая все требования конфиденциальности и защиты персональных данных.

Таким образом, стратегия монетизации проекта включает freemium-модель, партнерские программы, корпоративные решения, SaaS-подписки и интеграцию рекламы, что позволит не только окупить затраты, но и получить устойчивый доход для дальнейшего развития продукта.

ЗАКЛЮЧЕНИЕ

В ходе выполнения проекта было разработано веб-приложение «Персональный финансовый менеджер», предназначенное для анализа финансовых транзакций, управления личными целями и формирования отчетов. Приложение позволяет пользователям эффективно отслеживать свои расходы, планировать бюджет и повышать финансовую грамотность, что особенно актуально среди молодой аудитории в возрасте 16–40 лет.

Проект реализован с использованием современных технологий: React для клиентской части, Node.js и Express для серверной логики, PostgreSQL в качестве основной базы данных и Redis для кэширования. Благодаря выбранному стеку технологий обеспечивается высокая производительность, масштабируемость и безопасность приложения. Также была организована система CI/CD с использованием GitHub Actions, что позволяет оперативно обновлять и развёртывать приложение.

Особое внимание уделено удобству использования — интерфейс приложения интуитивно понятен даже для новичков. В отличие от аналогов, таких как «Кошелёк», «Дребеденьги» и Mint, наше решение предлагает более простой подход к учету транзакций, минималистичный дизайн и возможность работы в веб-браузере без необходимости установки мобильного приложения.

На текущем этапе реализованы базовые функции: добавление и редактирование транзакций, категоризация доходов и расходов, создание финансовых целей и формирование еженедельных отчетов. Однако потенциал развития проекта гораздо шире. Перспективными направлениями являются создание мобильных версий под Android и iOS, внедрение многоязычности и мультивалютности, расширение поддержки банковских API и интеграция искусственного интеллекта для персонализированных рекомендаций.

С точки зрения монетизации проект предусматривает использование freemium-модели, партнерских программ с банками, корпоративных решений и рекламы. Это позволит не только покрыть расходы на дальнейшее развитие

продукта, но и обеспечить его устойчивое развитие на рынке цифровых сервисов.

Таким образом, проект «Персональный финансовый менеджер» представляет собой современное и удобное решение для управления личными финансами, которое имеет хорошие перспективы для масштабирования и коммерциализации. Его реализация способствует популяризации финансовой грамотности и предоставляет пользователям инструменты для осознанного управления своими средствами.

ЛИТЕРАТУРА

1. ГОСТ 19.002-80 Схемы алгоритмов и программ. Правила выполнения [Текст] – Введ. с 01.07. 1981 г. М.: Изд-во стандартов, 1981. – 9 с.
2. ГОСТ 19.003-80 Схемы алгоритмов и программ. Обозначение условные графические [Текст] – Введ. с 01.07. 1981 г. М.: Изд-во стандартов, 1981. – 9 с.
3. Оформление выпускной квалификационной работы на соискание квалификационного уровня «Магистр» («Бакалавр»): методические рекомендации. / сост. Бержанский В.Н., Дзедолик И.В., Полулях С.Н. – Симферополь: КФУ им. В.И.Вернадского, 2019. – 31 с.

ПРИЛОЖЕНИЕ 1

ТЕХНИЧЕСКОЕ ЗАДАНИЕ НА ВЫПОЛНЕНИЕ РАБОТ ПО ПРОЕКТУ

1. Наименование программного продукта: Web-приложение для анализа финансовых транзакций и управления личными целями, позволяющее пользователям отслеживать свои расходы, планировать бюджет и получать еженедельные отчеты о финансовом состоянии.

2. Краткая характеристика области применения программного продукта:

Программный продукт предназначен для индивидуальных пользователей, желающих эффективно управлять своими финансами. Оно поможет в анализе транзакций, составлении бюджета и достижении финансовых целей, что повысит финансовую осведомленность и дисциплину.

3. Стек технологий: React, JWT, PostgreSQL, Node.js, MongoDB, GitHub, git, Nginx, Redis, Jira.

4. Каналы распространения и доставки: Пользователи узнают о продукте через социальные сети, блоги и специализированные сайты о финансах. Исходный код будет размещен на GitHub, а готовой продукт доступен по ссылке.

5. Аналоги\Конкуренты:

1) Кошелёк

Ссылка: <https://koshelek.ru>

Плюсы: - Простой и интуитивно понятный интерфейс. - Бесплатное использование. - Напоминания о платежах и планирование бюджета.

Минусы: - Нет автоматической синхронизации с банками. - Ограниченная аналитика. - Только мобильное приложение (нет веб-версии).

2) Дребеденьги

Ссылка: <https://drebedengi.ru>

Плюсы: - Поддержка совместного использования (например, для семьи). - Гибкие отчеты и аналитика. - Интеграция с банками.

Минусы: - Платная подписка для доступа к основным функциям. - Устаревший дизайн. - Сложность в настройке для новичков.

3) Mint

Ссылка: <https://mint.com>

Плюсы: - Автоматическая синхронизация с банками: Позволяет автоматически импортировать транзакции и отслеживать расходы. - Удобная аналитика: Подробные отчеты и графики для анализа финансов. - Бесплатное использование: Основные функции доступны без подписки.

Минусы: - Ограниченная поддержка за пределами США: Не все банки и финансовые учреждения поддерживаются. - Реклама: Бесплатная версия содержит рекламу, которая может раздражать пользователей. - Сложный интерфейс: Для новичков может быть сложно разобраться во всех функциях.

6. Отличие от аналогов: Наше приложение будет включать возможность интеграции с банковскими API для автоматического импорта транзакций и анализа, функцию составления индивидуальных финансовых целей, а также удобный и минималистичный интерфейс.

7. Целевая аудитория: Предполагаемая аудитория — молодые люди в возрасте 16-40 лет, заинтересованные в управлении своими средствами и улучшении финансовой грамотности.

8. Потенциал развития проекта: создание мобильного приложения, перевод интерфейса на иностранные языки, мультивалютность, расширенная аналитика и ИИ-рекомендации.

9. Основные технические параметры, определяющие количественные и качественные характеристики: покрытие тестами на уровне 90%, нагрузка не менее 100 запросов в секунду, скорость загрузки страницы не более 2 секунд.

10. Перечень работ, выполняемых в рамках проекта: - Создание макета; - Разработка frontend и backend; - Интеграция с банковскими API; - Проведение тестирования; - Развертывание CI/CD.

11. Отчетность по проекту:

- Техническое задание на выполнение работ по проекту;
- Календарный план работ по проекту;

- Матрица распределения ответственности по проекту;
- Диаграммы Ганта – 6 шт;
- АКТ о выполнении работ по проекту;
- Пояснительная записка к проекту;
- Видео-демонстрация работы программного продукта;
- Презентационные материалы.

12. Сроки выполнения работ: 16.03.2024 – 08.06.2024.